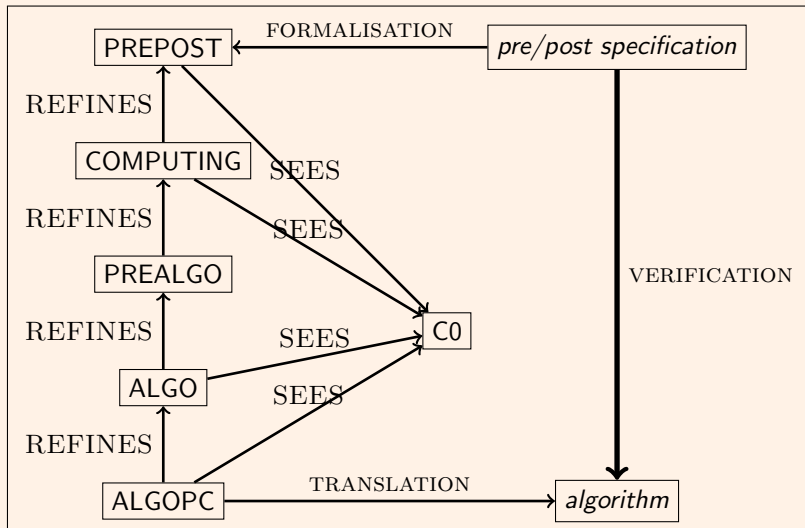


General Summary

- 1 Limits of the Inductive Paradigm
- 2 The Call-as-Event Paradigm
- 3 Introduction
- 4 Call as event
 - Main idea
 - Justification
- 5 Transformations of Event B Models into programs
 - Three main possible transformations
 - Integrated Development Framework
- 6 Case study : the binary search problem
- 7 Deriving the Recursive Program
 - Three possible events
 - Transforming the recursive algorithm
 - In the Spec# world
- 8 Case studies
 - Binomial coefficients
 - Sorting
 - Floyd's algorithm
 - Developing sequential algorithms
- 9 Summary

- 1 Limits of the Inductive Paradigm
- 2 The Call-as-Event Paradigm
- 3 Introduction
- 4 Call as event
- 5 Transformations of Event B Models into programs
- 6 Case study : the binary search problem
- 7 Deriving the Recursive Program
- 8 Case studies
- 9 Summary



CONTEXT $C0$

SETS

U

CONSTANTS

$x, v, d0, f, D$

AXIOMS

$axm1 : x \in \mathbb{N}$

$axm25 : D \subseteq U$

$axm24 : f \in D \rightarrow D$

$axm23 : d0 \in D$

$axm2 : v \in \mathbb{N} \rightarrow D$

$axm3 : v(0) = d0$

$axm4 : \forall n. n \in \mathbb{N} \Rightarrow v(n+1) = f(v(n))$

$th1 : Q(d0, d) \equiv (d = v(x))$

- the sequence v expresses the post-condition $Q(d_0, d)$ with the precondition $P(d_0)$.
- $Q(d_0, d)$ is equivalent to $d = v(x)$.
- The theorem $th1$ should be proved in the context C0. he

```
MACHINE PREPOST
SEES C0
VARIABLES
  r
INVARIANTS
  inv1 : r ∈ D
EVENTS
INITIALISATION
  BEGIN
    act1 : r ∈ D
  END
EVENT computing
  BEGIN
    act1 : r := v(x)
  END
END
```

- The theorem *th1* is validating the definition of the result *r* to compute.
- The event computing is expressing the *contract* of the given problem.
- it by a very simple problem that is the computation of the function n^2 using the addition operator.

EVENT INITIALISATION

BEGIN

$act1 : r \in D$

$act3 : vv := \{0 \mapsto d0\}$

$act5 : k := 0$

END

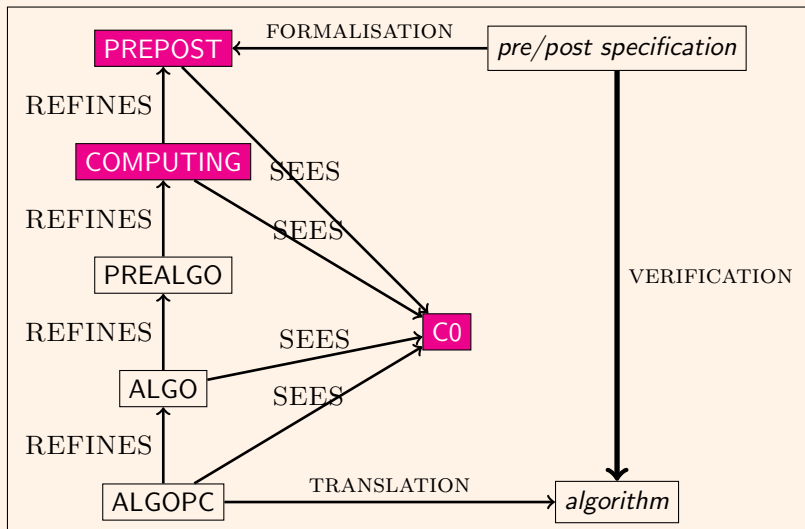
INITIALISATION is initializing the variables with respect to the initial values of the sequences of the context.

```
EVENT computing
  REFINES computing,
  WHEN
     $grd1 : x \in dom(vv)$ 
  THEN
     $act1 : r := vv(x)$ 
  END
END
```

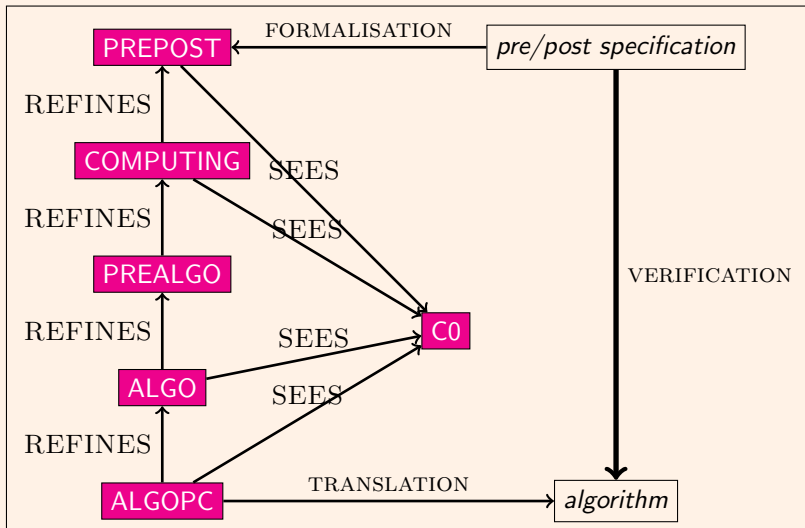
computing is imply
observing that the re-
sult is computed *simu-
lating* the sequence
 vv .

```
EVENT step
  WHEN
     $grd1 : x \notin dom(vv)$ 
  THEN
     $act2 : vv(k + 1) := f(vv(k))$ 
     $act4 : k := k + 1$ 
  END
```

step is *simulating* the computation of the values of the sequence vv as a model computation.



- **PREALGO** : adding new variables for pointing out the necessary values to store cvv
- **ALGO** : hiding the model variables storing the unnecessary values of sequence vv
- **ALGOPC** ; adding control variable c



Listing 1 – Function derived from pattern for the sequence v

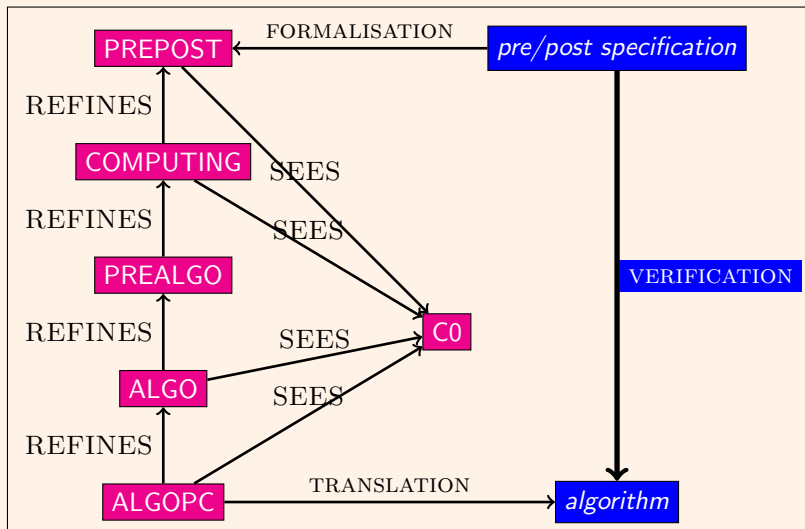
```
type(D)  f(int x)
{int    r, k, cv, or, ok, ocv;
  r=0;k=0;cv=0;or=0;ok=k;ocv=cv;
  while (k<x)
  {
    ok=k;ocv=cv;
    k=ok+1;
    cv=f(ocv);
  }
  r=cv;  return(r);}
```

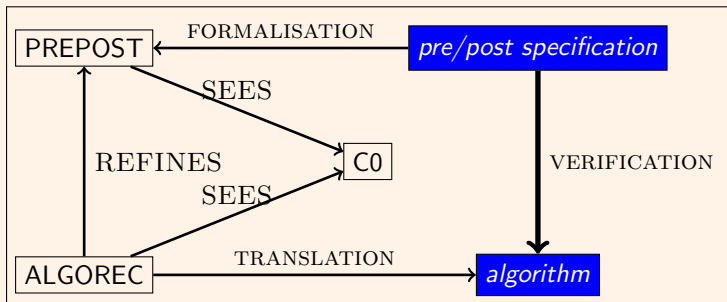

Listing 2 – Function derived from pattern power3

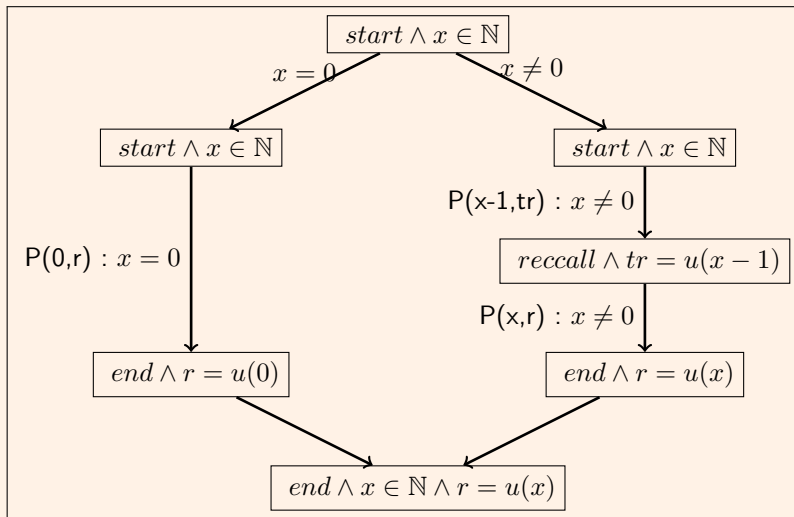
```
#include <limits.h>
/*@ requires 0 <= x;
    requires x*x*x <= INT_MAX ;
    ensures \result == x*x*x;
*/
int power3(int x)
{ int r, ocz, cz, cv, cu, ocv, cw, ocw, ct, oct, ocu, k, ok;
  cz=0; cv=0; cw=1; ct=3; cu=0; ocw=cw; ocz=cz;
  oct=ct; ocv=cv; ocu=cu; k=0; ok=k;
  /*@ loop invariant cz == k*k*k;
    @ loop invariant cu == k;
    @ loop invariant cv+ct==3*(cu+1)*(cu+1);
    @ loop invariant cz+cv+cw==3*(cu+1)*(cu+1)*(cu+1);
    @ loop invariant cv== 3*cu*cu;
    @ loop invariant cw == 3*cu+1;
    @ loop invariant k <= x;
    @ loop assigns ct, oct, cu, ocu, cz, ocv, k, cv, cw, r, ok;
    @ loop assigns ocv, ocw; */
  while (k<x)
  {
    ocv=cz; ok=k; ocv=cv; ocw=cw; oct=ct; ocu=cu;
    cz=ocz+ocv+ocw;
    cv=ocv+oct;
    ct=oct+6;
    cw=ocw+3;
    cu=ocu+1;
    k=ok+1;
  }
  r=cz; return (r); }
```

Summary for proof obligations

Name	Total	Automatic	Interactive
ex-induction	40	36	4
C0	2	0	2
PREPOST	4	4	0
COMPUTING	16	14	2
PREALGO	9	9	0
ALGO	6	6	0
ALGOPC	3	3	0







- **EVENT** $\text{rec}\% \text{PROC}(h(x), y) \% P(y)$ is simply simulating the recursive call of the same function.
- The invariant is defined in a simpler way by analysing the inductive structure and a control variable is introduced for structuring the inductive computation.

EVENT

e

WHERE $\ell = \ell_1$ $g_{\ell_1, \ell_2}(x)$ **THEN** $\ell := \ell_2$ $x := f_{\ell_1, \ell_2}(x)$ **END****EVENT** $\text{rec}\% \text{PROC}(h(x), y) \% P(y)$ **ANY** y **WHERE** $\ell = \ell_1$ $g_{\ell_1, \ell_2}(x, y)$ **THEN** $\ell := \ell_2$ $x := f_{\ell_1, \ell_2}(x, y)$ **END****EVENT** $\text{call}\% \text{APROC}(h(x), y) \% P(y)$ **ANY** y **WHERE** $\ell = \ell_1$ $g_{\ell_1, \ell_2}(x, y)$ **THEN** $\ell := \ell_2$ $x := f_{\ell_1, \ell_2}(x, y)$ **END**

VARIABLES

r, c, tr

INVARIANTS

$inv1 : r \in \mathbb{N}$

$inv2 : c = end \Rightarrow r = n * n$

$inv3 : c = callrec \Rightarrow n \neq 0$

$inv4 : c = callrec \Rightarrow tr = (n - 1) * (n - 1)$

$inv5 : c \in C$

$inv6 : tr \in \mathbb{N}$

$inv7 : c = end \Rightarrow r = n * n$

$inv8 : c = end \wedge n \neq 0$

$\Rightarrow tr = (n - 1) * (n - 1) \wedge r = tr + 2 * (n - 1) + 1$

$inv9 : c = callrec \Rightarrow n * n = tr + 2 * (n - 1) + 1$

EVENT INITIALISATION

BEGIN

$act1 : r := 0$

$act2 : c := start$

$act3 : tr : \in \mathbb{N}$

END

EVENT square0

REFINES square($n;r$)

WHEN

$grd1 : c = start$

$grd2 : n = 0$

THEN

$act1 : c := end$

$act2 : r := 0$

END

EVENT squaren

REFINES square($n;r$)

WHEN

$grd1 : c = callrec$

THEN

$act1 : r := tr + 2 * (n - 1) + 1$

$act2 : c := end$

END

EVENT rec%square($n-1;tr$)

WHEN

$grd1 : c = start$

$grd2 : n \neq 0$

THEN

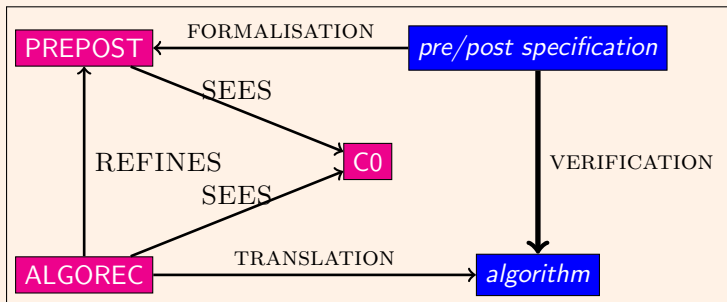
$act1 : c := callrec$

$act2 : tr := (n - 1) * (n - 1)$

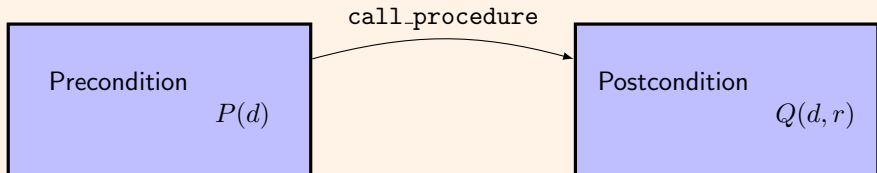
END

Summary for proof obligations

Name	Total	Automatic	Interactive
cae-square	34	32	2
square0	3	2	1
specsquare	2	2	0
square	29	28	1



- Proofs are easier and simpler
- Invariant is simple to find
- Translation is automatic
- Students are not happy with it and tools are not always set for this verification.



- `call_procedure` is simulated by the event `call_procedure` :

```
EVENT CALL_PROCEDURE
WHEN
   $P(d)$ 
THEN
   $r : Q(d, r')$ 
END
```

- $P(d)$ is specifying the inputs of the procedure.
- $Q(d, r)$ is expressing the relation between the inputs d and the results r .

- A pre/post specification is a pair of assertions $(pre, post)$
- $(pre, post)$ is defining a *transformation* over variables that is correct with respect to *HOARE* triples :

$$\{pre\} transformation \{post\}$$

- **Problem** : Design of a program `PROG` simulating the *transformation* with respect to $(pre, post)$:

$$\{pre\} PROG \{post\}$$

- **Idea** : Writing a *one-shot* relation simulating the relation over *flexible* variables :

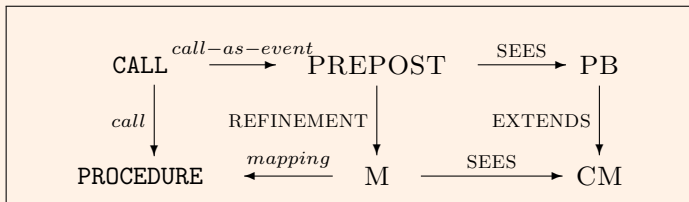
$$\forall x, y. pre(x) \wedge R(x, y) \Rightarrow post(x, y)$$

- **event** : Expressing the relation :

```
EVENT  PROG
WHEN   pre(x)
THEN   y : |post(x, y')
END
```

```
PROCEDURE  $\text{PROC}(x; \text{VAR } y)$   
PRECONDITION  $P(x)$   
POSTCONDITION  $Q(x, y)$ 
```

- Using the design-by-contract approach
- Introduction of concepts of programming (call-by-value,...) and of modelling (constants, axioms, ...)
- Using diagrams to improve the communication with students through definitions (dynamic programming)
- Organizing the global interactions between modelling and proving.



- CALL is the call of the PROCEDURE
- PREPOST is the machine containing the events stating the pre- and postconditions of CALL and PROCEDURE, and M is the refinement machine of PREPOST, with events including control points defined in CM.
- The *call-as-event* transformation produces a model PREPOST and a context PB from CALL.
- The *mapping* transformation allows us to derive an algorithmic procedure that can be mechanized.
- PROCEDURE is a node corresponding to a procedure derived from the refinement model M. CALL is an instantiation of PROCEDURE using parameters x and y .
- M is a refinement model of PREPOST, which is transformed into PROCEDURE by applying structuring rules. It may contain events corresponding to calls of other procedures.

- Two domains D_1 and D_2 that define the global set of possible values of the current system for its inputs and outputs.
- A list of constants x that specify the input of the system under development $\tilde{P}$, which is the set of values for x defining the precondition, and $\tilde{Q}$, which is a binary relation over $D_1 \times D_2$ defining the postcondition for the problem.
- A list of axioms that assigns types to constants and adds knowledge to the RODIN environment.
- The theorems that state the existence of solutions with respect to preconditions : for instance, theorem *th1.i* states that there is always a solution y when the input value x satisfies the precondition P .

CONTEXT PB

SETS

D_1, D_2

CONSTANTS

$x, n, \tilde{P}, \tilde{Q}$

AXIOMS

$axm1 : x \in D_1$

x belongs to a general set of the problem domain

$axm2 : n \in \mathbb{NAT1}$

n is the number of different possible preconditions, n is not equal to 0 and it is generally 1

$axm3 : \tilde{P} \in 1..n \longrightarrow \mathbb{P}(D_1)$

$\tilde{P}$ is a collection of sets defining the precondition

$axm4 : \tilde{Q} \subseteq D_1 \times D_2$

$\tilde{Q}$ is a binary relation over $D_1 \times D_2$ defining the postcondition

$axm5 : x \in \bigcup_{i \in 1..n} \tilde{P}(i)$

x is supposed to satisfy the precondition $P(i)$

THEOREMS

$th1.i : \forall a \cdot a \in \tilde{P}(i) \Rightarrow (\exists b \cdot a \mapsto b \in \tilde{Q})$

there is at least one solution for each data x satisfying the precondition P_i

END

```

EVENT calli
WHEN
     $P_i(x)$ 
THEN
     $y : |(Q(x, y'))$ 
END

```

- For each call CALL_i , y is set to a value satisfying $Q(x, y)$ when x satisfies $P(x)$.
- The variable y is initialized by a value satisfying the assertion $\text{Init}(y, x, D_1, D_2)$.
- The machine PREPOST, which sees and uses the data defined in the CONTEXT PB and which encapsulates the n cases corresponding to the n preconditions.

MACHINE PREPOST

SEES PB

VARIABLES

y

INVARIANTS

$inv1 : y \in D_2$

EVENTS

INITIALISATION

BEGIN

$act1 : y \in D_2$

END

...

EVENT $call_i$

WHEN

$P_i(x)$

THEN

$y : |(Q(x, y'))$

END

...

END

- $call_i$ is corresponding to the i th call.
- The relation defined by Q may be non computable
- The refinement can be leading to an non computable solution.

Suppose that PB is a context and PREPOST is a machine corresponding to a problem stating calls of a procedure. Suppose that the following diagram is validated :

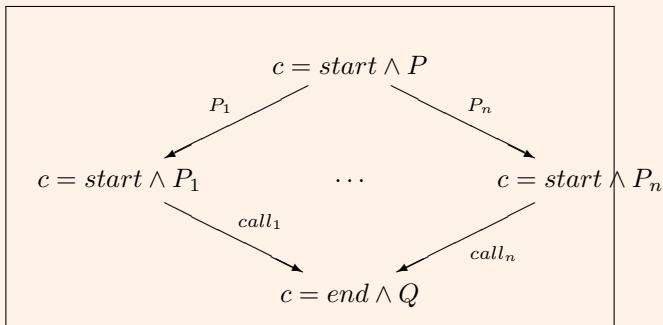


We assume that the preconditions are defined by P , i.e., $P_1, \dots, P_n$, and the postcondition is defined by Q .
Then for any i in $1..n$, PREPOST satisfies $P_i(x, y) \rightsquigarrow Q(x, y)$.

A refinement diagram for M , P , and Q over L and A is an acyclic labelled graph over A with labels from G or E satisfying the following rules.

- There is a unique input node P with at least one outgoing arrow.
- There is a unique output node Q with no outgoing arrows.
- If R is related to S by a unique arrow labelled $e \in E$, then
 - ▶ It satisfies the property $R \leadsto S$
 - ▶ $\forall c, x, c', x'. R(c, x) \wedge I(M)(c, x) \wedge BA(e)(c, x, c', x') \Rightarrow S(c', x')$
 - ▶ $\forall c, x. R(c, x) \wedge I(M)(c, x) \Rightarrow \exists c', x'. BA(e)(c, x, c', x')$
 - ▶ If $R \equiv c = l1 \wedge A(x)$ and $S \equiv c = l2 \wedge B(x)$, then $l1 \neq l2$ and $\ell(R)=l1, \ell(S)=l2$.
- If R is related to $S_1, \dots, S_p$, then
 - ▶ Each arrow R to S_i is labelled by a guard $g_i \in G$.
 - ▶ For any i in $1..p$ the following conditions hold.
$$\left(\begin{array}{l} R \wedge I(M) \wedge g_i(x) \Rightarrow S_i \\ \forall j. j \in 1..p \wedge j \neq i \wedge R \wedge I(M) \wedge g_i(x) \Rightarrow \neg g_j(x) \end{array} \right)$$
 - ▶ $R \wedge I(M) \Rightarrow \exists i \in 1..p. g_i$.
- For each $e \in E$, there is only one instance of e in the diagram.

We use $PRE(D)$ for P and $POST(D)$ for Q .



- 1 Limits of the Inductive Paradigm
- 2 The Call-as-Event Paradigm
- 3 Introduction
- 4 Call as event
- 5 Transformations of Event B Models into programs**
- 6 Case study : the binary search problem
- 7 Deriving the Recursive Program
- 8 Case studies
- 9 Summary

```
EVENT e
WHEN
   $\ell = \ell_1$ 
   $g_{\ell_1, \ell_2}(x)$ 
THEN
   $\ell := \ell_2$ 
   $x := f_{\ell_1, \ell_2}(x)$ 
END1
```

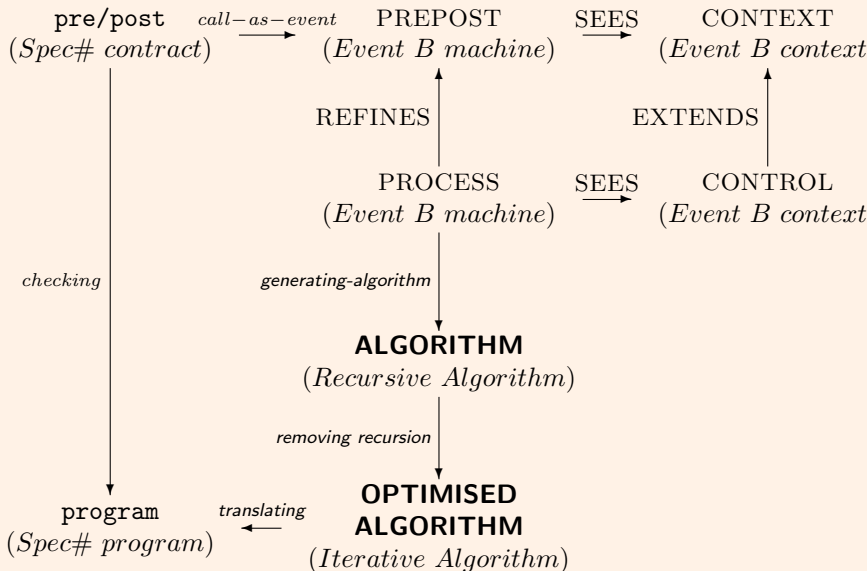
- If the event e is a basic event controlling the state of the variable x , guarded by $g_{\ell_1, \ell_2}(x)$ and modified by the assignment $x := f_{\ell_1, \ell_2}$ where f is a function, the event e takes the form below.
- the function f_{ℓ_1, ℓ_2} is implementable.
- If the event e labels the link $\ell_1 \xrightarrow{e} \ell_2$ then the statement act_{ℓ_2} is defined as
WHEN $g_{\ell_1, \ell_2}(x)$ **THEN** $x := f_{\ell_1, \ell_2}(x)$.

```
EVENT rec%PROC(h(x),y)%P(y)
ANY y
WHEN
   $\ell = \ell_1$ 
   $g_{\ell_1, \ell_2}(x, y)$ 
THEN
   $\ell := \ell_2$ 
   $x := f_{\ell_1, \ell_2}(x, y)$ 
END1
```

- The definition of the event e is not executable and the translation is driven by instances of the control variable ℓ in the guard (as $\ell = \ell_1$) and in the assignment ($\ell := \ell_2$).
- The statement act_{ℓ_2} is therefore defined as : $\text{PROC}(h(x), y)$.
- The choice of the event name is the responsibility of the writer of the EVENT B models, who must identify the case corresponding to a recursive call.

```
EVENT call%APROC(h(x),y)%P(y)
ANY y
WHEN
   $\ell = \ell_1$ 
   $g_{\ell_1, \ell_2}(x, y)$ 
THEN
   $\ell := \ell_2$ 
   $x := f_{\ell_1, \ell_2}(x, y)$ 
END1
```

- the event e can be transformed into a call of another procedure.
- The call is expressed by an event e , which we name $call\%APROC(h(x), y)\%P(y)$ and the statement act_{ℓ_2} is defined as $APROC(h(x), y)$.
- APROC is defined or to be defined in another framework.



ANY t **WHERE** $G(t, x)$ **THEN** $x : |(R(x, x', t))$ **END**

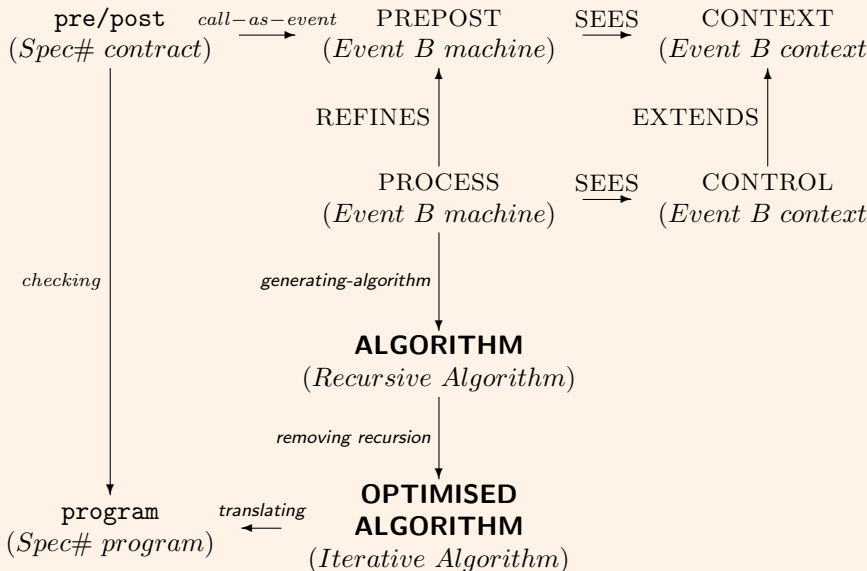
(t is a local parameter and the event actions establish $x : |(R(x, x', t))$)
($\exists t \cdot (G(t, x) \wedge R(x, x', t))$)

```
MACHINE specsquare
SEES square0
VARIABLES
   $r$ 
INVARIANTS
   $inv1 : r \in \mathbb{N}$ 
EVENTS
EVENT INITIALISATION
  BEGIN
     $act1 : r := 0$ 
  END
EVENT square_computing
  BEGIN
     $act1 : r := n * n$ 
  END
END
```

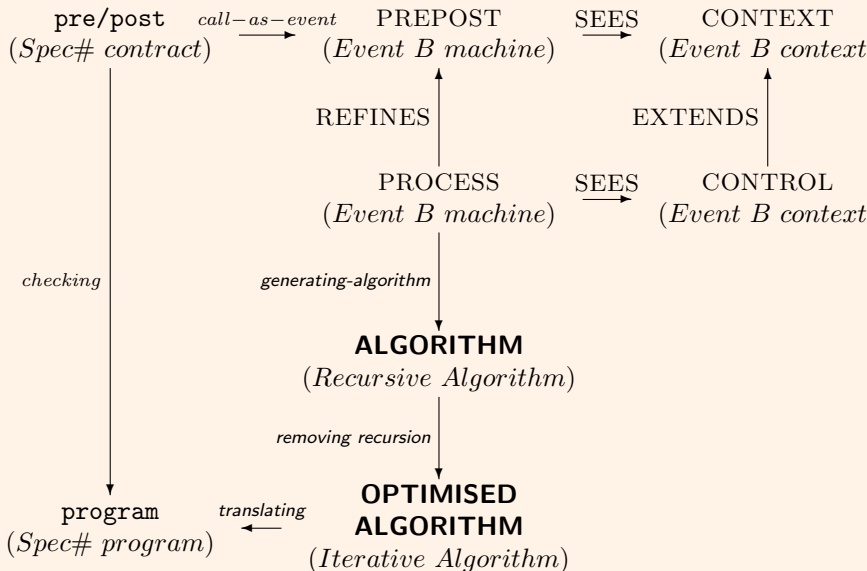
```
CONTEXT square0
CONSTANTS
   $n$ 
AXIOMS
   $axm1 : n \in \mathbb{N}$ 
END
```

- *square0* is a context defining properties of a natural number n
- *specsquare* is a machine with an event *square_computing* computing the square function for n and assigning the value to r .
- The **SEES** clause related the context and the machine.


```
public static void sortArray(int[] !st, int[] !pi)
  requires st != pi && st.Length == pi.Length;
  requires forall{int i in (0:st.Length); st[i] == pi[i]};
  modifies st[*];
  ensures forall{int j in (1:st.Length); (st[j-1] <= st[j])};
  ensures forall{int w in (0:st.Length);
    (count{int v in (0:st.Length); st[v] == pi[w]}
    == count{int u in (0:st.Length); pi[u] == pi[w]})};
```



- The EVENT B machine PREPOST contains events, which have the same contract as that expressed in the original pre/post contract. This machine SEES the EVENT B CONTEXT, which expresses static information about the machine.
- The EVENT B machine PROCESS refines PREPOST generating a concrete specification that satisfies the contract. This machine SEES the EVENT B context CONTROL, which adds control information for the new machine.
- The labelled actions REFINES, SEES and EXTENDS, are supported by the RODIN platform and are checked *completely* using the proof assistant provided by RODIN.
- Transformation of an EVENT B machine into a concrete recursive algorithm (represented by the arrow labelled *generating-algorithm*).
- Transformation of this recursive algorithm into its equivalent partially annotated and iterative algorithm (represented by the arrow labelled *removing recursion*).



Our integrated development framework for implementing abstract EVENT B models brings together the strengths of the refinement based approaches and verification based approaches to software development :

- ① Splitting the abstract specification to be solved into its component specifications.
- ② Refining these specifications into a concrete model using EVENT B and the RODIN platform.
- ③ Transforming the concrete model into recursive and iterative algorithms that can be directly implemented as real source code.
- ④ Verifying the iterative algorithm in the automatic program verification environment of Spec#.

- 1 Limits of the Inductive Paradigm
- 2 The Call-as-Event Paradigm
- 3 Introduction
- 4 Call as event
- 5 Transformations of Event B Models into programs
- 6 **Case study : the binary search problem**
- 7 Deriving the Recursive Program
- 8 Case studies
- 9 Summary

$$\begin{array}{l} \text{PROCEDURE } \textit{binsearch}(t, \textit{val}, \textit{lo}, \textit{hi}, \textit{ok}, \textit{result}) \\ \text{PRE } \left(\begin{array}{l} t \in 0..t.Length \longrightarrow \mathbb{N} \\ \forall k.k \in \textit{lo}..\textit{hi} - 1 \Rightarrow t(k) \leq t(k+1) \\ \textit{val} \in \mathbb{N} \\ \textit{l}, \textit{h} \in 0..t.Length \\ \textit{lo} \leq \textit{hi} \end{array} \right) \\ \text{POST } \left(\begin{array}{l} \textit{ok} = \textit{TRUE} \Rightarrow t(\textit{result}) = \textit{val} \\ \textit{ok} = \textit{FALSE} \Rightarrow (\forall i.i \in \textit{lo}..\textit{hi} \Rightarrow t(i) \neq \textit{val}) \end{array} \right) \end{array}$$

The two possible resulting *calls* to the procedure *binsearch*(*t, val, lo, hi; ok, result*) :

- **EVENT** find is $\text{binsearch}(t, \text{val}, lo, hi; ok, result)$ with $ok = TRUE$
- **EVENT** fail is $\text{binsearch}(t, \text{val}, lo, hi; ok, result)$: with $ok = FALSE$

```
EVENT find
  ANY  $j$ 
  WHERE
     $grd1 : j \in lo .. hi$ 
     $grd2 : t(j) = val$ 
  THEN
     $act1 : ok := TRUE$ 
     $act2 : result := j$ 
  END
```

```
EVENT fail
  WHEN
     $grd1 : \forall k \cdot k \in lo .. hi \Rightarrow t(k) \neq val$ 
  THEN
     $act1 : ok := FALSE$ 
  END
```

- The two events form the machine called *binsearch1* (which corresponds to the PREPOST machine).
- The machine is refined to obtain *binsearch2* (which corresponds to PROCESS).
- This refined machine contains a new control variable, l , which *simulates* how the binary search is achieved.

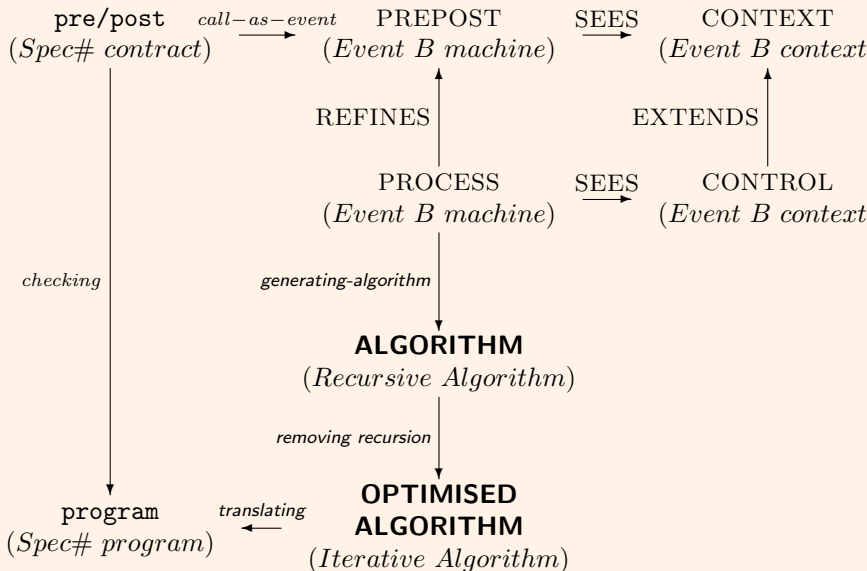
$$\textcircled{1} \left(\begin{array}{l} l = \textit{start} \\ lo = hi \\ t(lo) = val \end{array} \right) \xrightarrow{m_1} \left(\begin{array}{l} l = end \\ lo = hi \\ ok = TRUE \wedge result = lo \end{array} \right)$$

$$\textcircled{2} \left(\begin{array}{l} l = \textit{start} \\ lo = hi \\ t(lo) \neq val \end{array} \right) \xrightarrow{m_2} \left(\begin{array}{l} l = end \\ lo = hi \\ ok = FALSE \end{array} \right)$$

$$\textcircled{3} \left(\begin{array}{l} l = \textit{start} \\ lo < hi \end{array} \right) \xrightarrow{\textit{split}} \left(\begin{array}{l} l = \textit{middle} \\ lo < hi \\ mi = (lo + hi)/2 \end{array} \right)$$

$$\textcircled{4} \left(\begin{array}{l} l = \textit{middle} \\ lo < hi \\ mi = (lo + hi)/2 \\ val < t(mi) \end{array} \right) \xrightarrow{\textit{rec}(lo, mi-1, val, ok, result)} \left(\begin{array}{l} l = end \\ ok = TRUE \wedge t(result) = val \end{array} \right)$$

- 1
$$\left(\begin{array}{l} l = middle \\ lo < hi \\ val < t(mi) \end{array} \right) \xrightarrow{rec(lo, mi-1, val, ok, result)} \left(\begin{array}{l} l = end \\ \wedge ok = FALSE \wedge (\forall i. i \in lo..hi \Rightarrow t(i) \neq val) \end{array} \right)$$
- 2
$$\left(\begin{array}{l} l = middle \\ lo < hi \\ mi = (lo + hi)/2 \\ val = t(mi) \end{array} \right) \xrightarrow{m_3} \left(\begin{array}{l} l = end \\ ok = TRUE \wedge result = mi \end{array} \right)$$
- 3
$$\left(\begin{array}{l} l = middle \\ lo < hi \\ mi = (lo + hi)/2 \\ val > t(mi) \end{array} \right) \xrightarrow{rec(mi+1, hi, val, ok, result)} \left(\begin{array}{l} l = end \\ ok = TRUE \wedge t(result) = val \end{array} \right)$$
- 4
$$\left(\begin{array}{l} l = middle \\ lo < hi \\ mi = (lo + hi)/2 \\ val > t(mi) \end{array} \right) \xrightarrow{rec(mi+1, hi, val, ok, result)} \left(\begin{array}{l} l = end \\ \wedge ok = FALSE \wedge (\forall i. i \in lo..hi \Rightarrow t(i) \neq val) \end{array} \right)$$



- 1 Limits of the Inductive Paradigm
- 2 The Call-as-Event Paradigm
- 3 Introduction
- 4 Call as event
- 5 Transformations of Event B Models into programs
- 6 Case study : the binary search problem
- 7 Deriving the Recursive Program
- 8 Case studies
- 9 Summary


```

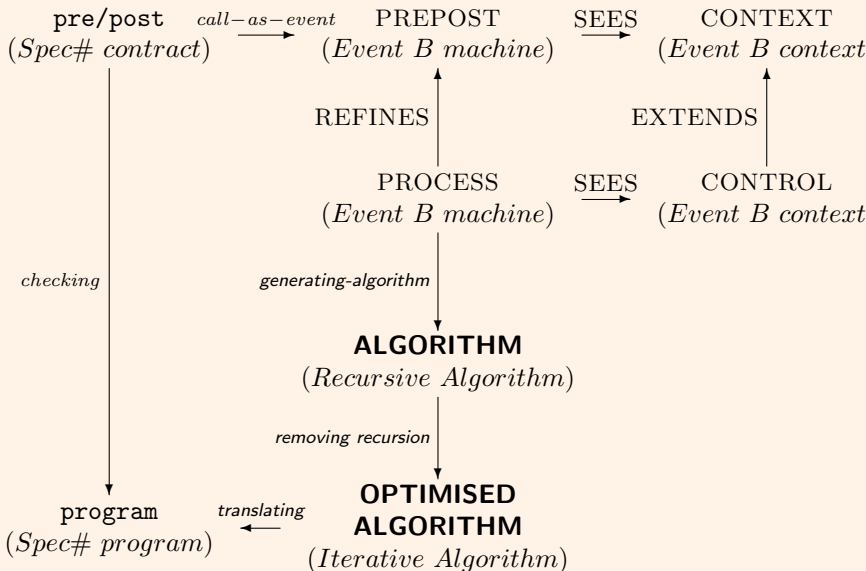
EVENT  $\text{rec}\% \text{PROC}(h(x), y) \% P(y)$ 
ANY  $y$ 
WHEN
     $\ell = \ell_1$ 
     $g_{\ell_1, \ell_2}(x, y)$ 
THEN
     $\ell := \ell_2$ 
     $x := f_{\ell_1, \ell_2}(x, y)$ 
END1

```

- The definition of the event e is not executable and the translation is driven by instances of the control variable ℓ in the guard (as $\ell = \ell_1$) and in the assignment ($\ell := \ell_2$).
- The statement act_{ℓ_2} is therefore defined as : $\text{PROC}(h(x), y)$.
- The choice of the event name is the responsibility of the writer of the EVENT B models, who must identify the case corresponding to a recursive call.

```
EVENT call%APROC(h(x),y)%P(y)
ANY y
WHEN
   $\ell = \ell_1$ 
   $g_{\ell_1, \ell_2}(x, y)$ 
THEN
   $\ell := \ell_2$ 
   $x := f_{\ell_1, \ell_2}(x, y)$ 
END1
```

- the event e can be transformed into a call of another procedure.
- The call is expressed by an event e , which we name $call\%APROC(h(x), y)\%P(y)$ and the statement act_{ℓ_2} is defined as $APROC(h(x), y)$.
- APROC is defined or to be defined in another framework.



- Correct by construction according to the Event B side
- Limits of the Spec# tools with verifying recursive program
- Transforming recursive algorithms into iterative algorithm
- Applying the Spec# tools

Theorem

The transformation is sound with respect to the pre and post specification.

```
PROCEDURE APROC( $x$ ; VAR  $y$ )
PRECONDITION  $P(x)$ 
POSTCONDITION  $Q(x, y)$ 
BEGIN
LOCAL VARIABLES  $z$ 
IF  $C(x)$  THEN
 $y := g(x)$ ;
ELSE
 $z := h(x, z)$ ;
IF  $D(x, z)$  THEN
 $y := f(x, z)$ 
ELSEIF  $E(x, z)$  THEN
APROC( $f_1(x), y$ )
ELSE
APROC( $f_2(x), y$ )
ENDIF
ENDIF
END
```

```
PROCEDURE BPROC( $x$ ; VAR  $y$ )
PRECONDITION  $P(x)$ 
POSTCONDITION  $Q(x, y)$ 
BEGIN
LOCAL VARIABLES  $z$ 
WHILE not  $C(x) \wedge$  not  $D(x, z)$  DO
 $z := h(x, z)$ ;
IF  $E(x, z)$  THEN
 $x := f_1(x)$ ;
ELSE
 $x := f_2(x)$ ;
ENDIF
ENDDO
IF  $C(x)$  THEN
 $y := g(x)$ ;
ELSEIF  $D(x, z)$  THEN
 $y := f(x, z)$ ;
ELSEIF  $E(x, z)$  THEN
 $y := f_1(x)$ ;
ELSE
 $y := f_2(x)$ ;
ENDIF
END
```

PROCEDURE `binsearch(t, val, lo, hi, ok, result)`

PROCEDURE `binsearch(t, val, lo, hi, ok, result)`

PRECONDITION $\left(\begin{array}{l} t \in 0..t.Length \rightarrow \mathbb{N} \\ \forall k. k \in lo..hi - 1 \Rightarrow t(k) \leq t(k+1) \\ val \in \mathbb{N} \wedge lo, hi \in 0..t.Length \wedge lo \leq hi \end{array} \right)$

POSTCONDITION $\left(\begin{array}{l} ok = TRUE \Rightarrow t(result) = val \\ ok = FALSE \Rightarrow (\forall i. i \in lo..hi \Rightarrow t(i) \neq val) \end{array} \right)$

BEGIN

WHILE *not* $\left(\begin{array}{l} lo = hi \wedge t(lo) = val \\ \vee lo = hi \wedge t(lo) \neq val \\ \vee lo < hi \wedge mi = (lo + hi)/2 \wedge t(mi) = val \end{array} \right)$ **DO**

$mi := (lo + hi)/2;$

$middle : \left\{ \left(\begin{array}{l} mi = (lo + hi)/2 \\ val < t(mi) \Rightarrow \forall k. k \in mi..hi \Rightarrow t(k) \neq val \\ val > t(mi) \Rightarrow \forall k. k \in lo..mi \Rightarrow t(k) \neq val \end{array} \right) \right\}$

IF $mi + 1 \leq hi \wedge val > t(mi)$ **THEN**

$lo := mi + 1$

ELSEIF $lo \leq mi - 1 \wedge val < t(mi)$ **THEN**

$hi := mi - 1$

ENDDO

IF $lo = hi \wedge t(lo) = val$ **THEN**

$result := lo; ok := true$

ELSEIF $lo = hi \wedge t(lo) \neq val$ **THEN**

$ok := false$

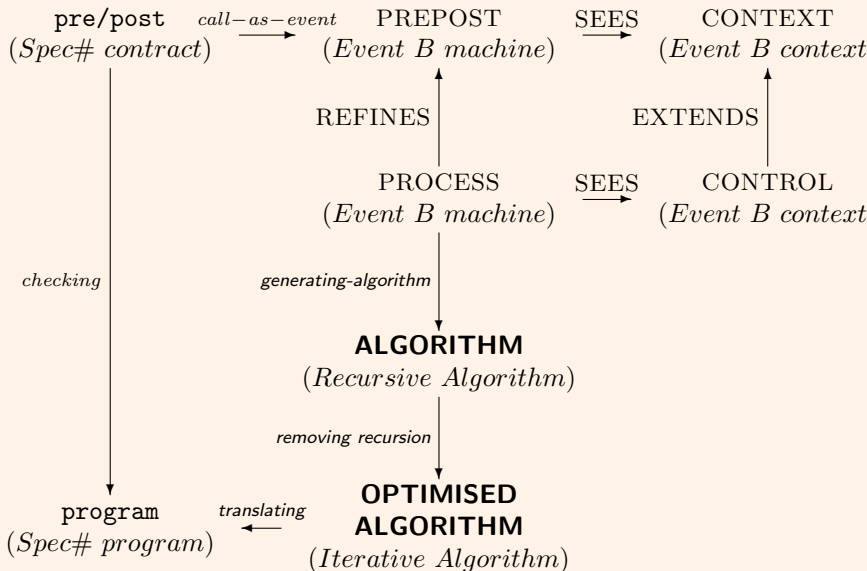
ELSEIF $lo < hi \wedge t(mi) = val$ **THEN**

$result := mi; ok := true$

ELSE $ok := false$

ENDIF

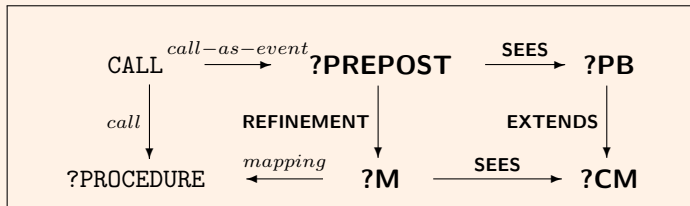
END



- This is almost a one-to-one mapping : returning a value of -1 when our iterative algorithm sets *OK* to *false* and returning the index where the value is found when our iterative algorithm sets *OK* to *true*.
- The algorithm verified as correct, in less than 2 seconds using the Spec# programming system (version 2011-10-03).
- No user interaction is required in the verification as all assertions required (preconditions, postconditions and loop invariants) have been generated as part of the refinement and transformation of the initial abstract specification into the final iterative algorithm.
- Prior to formalising our transformation rules, our initial attempt at writing this iterative C# program contained an error.
- This error in the loop body, was due to our omission to check that the values of $mi + 1$ and $mi - 1$ were within the array bounds before narrowing the search space.
- This error was immediately detected by the Spec# programming system. The automatic verification of the final program is available online at <http://www.rise4fun.com/SpecSharp/psP4>.

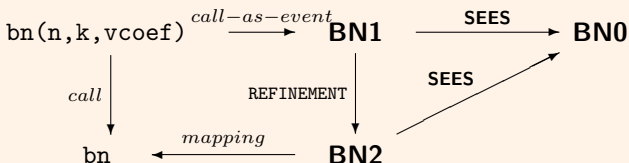
- Our integrated development framework indicates where our transformations are used for producing a program that is *correct-by-construction*.
- The translation of the PROCESS machine into a recursive algorithm is straightforward and removes the control variable used to relate events when generating the code.
- Our experience shows that our approach assists students in developing and understanding the tasks of software specification and verification.
- It also makes different forms of formal software development more accessible to the Software Engineers, helping them to build correct and reliable software systems.
- *EB2RC* : Update Site :
<http://www.cs.nuim.ie/~zcheng/EB2RC/TEB2RC/> :
<http://www.cs.nuim.ie/~zcheng/EB2RC> Zheng Cheng
[zcheng@cs.nuim.ie] and Rosemary Monahan
[rosemary.monahan@nuim.ie]

- Presenting the method : **Computing binomial coefficients**
- Illustrating one classical example : **Sorting by insertion**
- Using the dynamic programming : **Floyd's algorithm**



- **PB** : using textbooks for deriving axioms and theorems.
- **CALL** : using a programming language like JML or Spec#

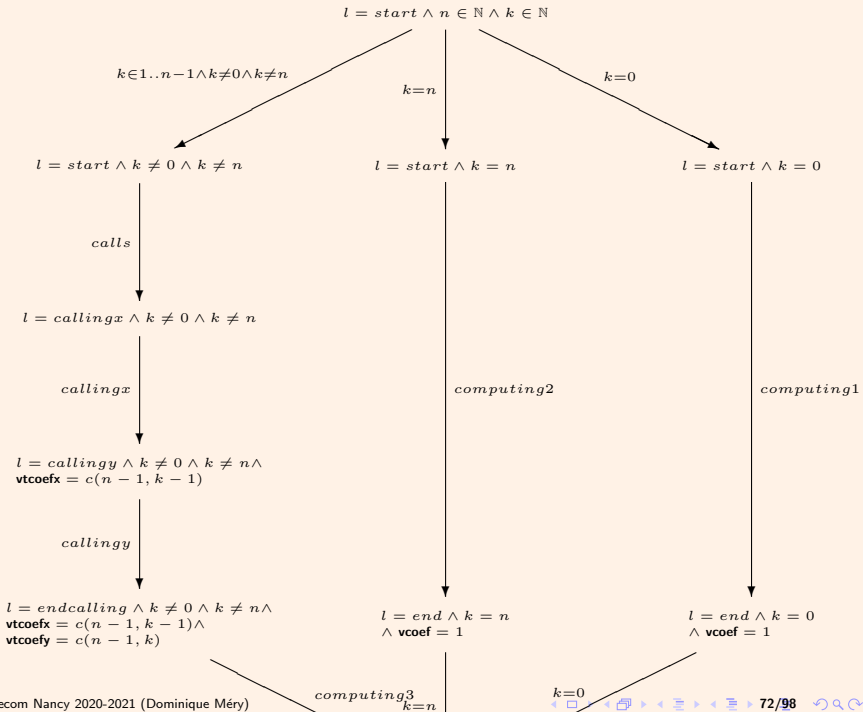
Problem 1 : Computing the binomial coefficients



- The computation of binomial coefficients is based on Pascal's triangle and we define it as a partial function c .
- Data n and k are defined in the context called $BN0$.
- The call $bn(n, k, vcoef)$ is translated as an event which is simply setting $vcoef$ to $c(n \mapsto k)$.
- The refinement $BN2$ produces a collection of events analysing the different steps of the computations required for computing the value of $c(n \mapsto k)$.

- Pascal's triangle provides a graphical guide for writing d 's definition into $BN0$.
- We have introduced another graphical structure for supporting the case analysis related to the values of n and k and the introduction of control flow.
- The world of mathematics is defining a value $c(n \mapsto k)$ and the world of computing will derive a process using c and its definition for producing the same value.

■ ■ ■





INVARIANTS

$inv1 : l \in LOC$

$inv3 : vtcoefx \in \mathbb{N}$

$inv4 : vtcoefy \in \mathbb{N}$

$inv5 : l \in \{callx, cally, endcalling\} \Rightarrow k \neq 0 \wedge n \neq 0 \wedge k < n$

$inv6 : l = cally \Rightarrow vtcoefx = c(n - 1 \mapsto k - 1)$

$inv7 : l = endcalling \Rightarrow vtcoefy = c(n - 1 \mapsto k) \wedge vtcoefx = c(n - 1 \mapsto k - 1)$

$inv8 : l = end \Rightarrow vcoef = c(n \mapsto k)$

- The refinement produces 42 proof obligations and 2 were manual. The other proof obligations are automatically discharged.

Teacher's note: The example is simple and the function c is easy to define. The invariant is built by analysing the expression of the computed value.

BINOMIAL0

SETS

LOC

CONSTANTS

n, k, c, start, end, callx, cally, endcalling

AXIOMS

axm1 : $n \in \mathbb{N}$

axm2 : $k \in \mathbb{N}$

axm3 : $k \leq n$

axm4 : $c \in 0 \dots n \times 0 \dots n \mapsto \mathbb{N}$

axm5 : $\text{dom}(c) = \{i \mapsto j \mid i \in 0 \dots n \wedge j \in 0 \dots n \wedge j \leq i\}$

axm6 : $\forall j. j \in 0 \dots n \Rightarrow c(j \mapsto 0) = 1$

axm7 : $\forall j. j \in 0 \dots n \Rightarrow c(j \mapsto j) = 1$

axm8 : $\text{LOC} = \{\text{start}, \text{end}, \text{callx}, \text{cally}, \text{endcalling}\}$

axm9 : $\text{start} \neq \text{end}$

axm10 : $\text{start} \neq \text{callx}$

axm11 : $\text{start} \neq \text{cally}$

axm12 : $\text{start} \neq \text{endcalling}$

axm13 : $\text{end} \neq \text{callx}$

axm14 : $\text{end} \neq \text{cally}$

axm15 : $\text{end} \neq \text{endcalling}$

axm16 : $\text{callx} \neq \text{cally}$

axm17 : $\text{callx} \neq \text{endcalling}$

axm18 : $\text{cally} \neq \text{endcalling}$

axm19 : $k \leq n$

axm20 : $\forall i, j. i \in 0 \dots n \wedge j \in 0 \dots n \wedge i < j \Rightarrow c(j \mapsto i) = c(j - 1 \mapsto i) + c(j - 1 \mapsto i - 1)$

END

```

MACHINE binomial1
SEES binomial0
VARIABLES
  vcoef
INVARIANTS
  myteminv1 vcoef ∈ EVENTS
INITIALISATION
  BEGIN
    act1 : vcoef : ∈ N
  END
EVENT computing
  BEGIN
    act1 : vcoef := c(n ↦ k)
  END
END

```


INITIALISATION

BEGIN

$act1 : vcoef \in \mathbb{N}$

$act2 : l := start$

$act4 : vtcoefx \in \mathbb{N}$

$act5 : vtcoefy \in \mathbb{N}$

END

EVENT computing1 **REFINES** *computing*

WHEN

$grd1 : k = 0$

$grd2 : l = start$

THEN

$act1 : vcoef := 1$

$act2 : l := end$

END

```

EVENT computing2 REFINES computing
WHEN
   $grd1 : l = start$ 
   $grd2 : k = n$ 
THEN
   $act1 : vcoef := 1$ 
   $act2 : l := end$ 
END

```

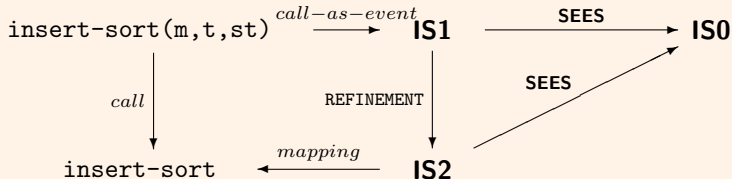
```
EVENT callingx
WHEN
  grd1 :  $l = start$ 
  grd2 :  $k \neq 0$ 
  grd3 :  $k \neq n$ 
THEN
  act1 :  $vtcoefx := c(n - 1 \mapsto k - 1)$ 
  act2 :  $l := cally$ 
END
EVENT callingy
WHEN
  grd1 :  $l = cally$ 
THEN
  act1 :  $vtcoefy := c(n - 1 \mapsto k)$ 
  act2 :  $l := endcalling$ 
END
```

```

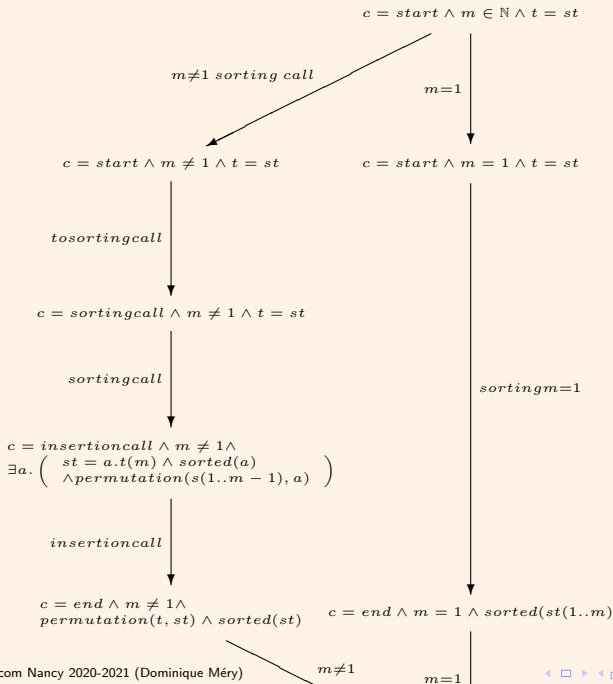
EVENT computing3 REFINES computing
WHEN
   $grd1 : l = endcalling$ 
THEN
   $act1 : vcoef := vtcoefx + vtcoefy$ 
   $act2 : l := end$ 
END
END

```

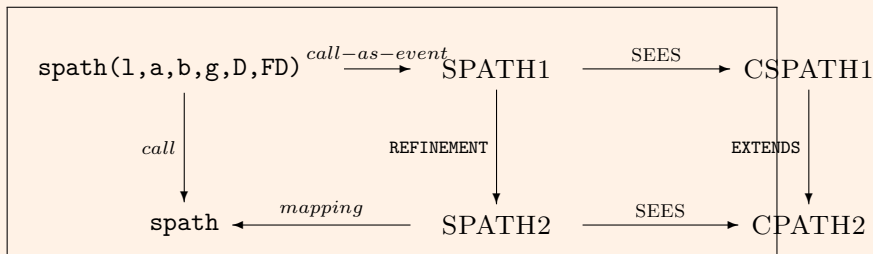
Problem 2 : Sorting by insertion



- The problem is to sort an array t between 1 and m , where $\text{dom}(t) = 1..n$ and $m \leq n$.
- The sorting can be done by sorting the array from 1 to $m - 1$ and then to insert the value $t(m)$ at the right position in $1..m$.
- The insertion event is considered as a call of procedure : the subproblem is solved in the same way by applying the guideline.



Problem 3 : Floyd's algorithm



- spath is built from events of SHORTESTPATH2
- FD states if the path exists
- D contains the cost of the minimal path, if it exists

Axioms for the partial function d

- $$axm5 : \forall i. i \in N \Rightarrow 0 \mapsto i \mapsto i \in dom(d) \wedge d(0 \mapsto i \mapsto i) = 0$$

- $$axm6 : \forall i, j, k. \left(\begin{array}{l} \left(\begin{array}{l} k - 1 \mapsto i \mapsto j \in dom(d) \\ \wedge (k - 1 \mapsto i \mapsto k \notin dom(d) \vee k - 1 \mapsto k \mapsto j \notin dom(d)) \end{array} \right) \\ \Rightarrow \\ \left(\begin{array}{l} k \mapsto i \mapsto j \in dom(d) \wedge d(k \mapsto i \mapsto j) = d(k - 1 \mapsto i \mapsto j) \end{array} \right) \end{array} \right)$$

- $$axm7 : \forall i, j, k. \left(\begin{array}{l} \left(\begin{array}{l} k - 1 \mapsto i \mapsto j \in dom(d) \\ \wedge k - 1 \mapsto i \mapsto k \in dom(d) \\ \wedge k - 1 \mapsto k \mapsto j \in dom(d) \\ \wedge d(k - 1 \mapsto i \mapsto j) \leq d(k - 1 \mapsto i \mapsto k) + d(k - 1 \mapsto k \mapsto j) \end{array} \right) \\ \Rightarrow \\ \left(\begin{array}{l} k \mapsto i \mapsto j \in dom(d) \\ \wedge d(k \mapsto i \mapsto j) = d(k - 1 \mapsto i \mapsto j) \end{array} \right) \end{array} \right)$$

- $$axm8 : \forall i, j, k. \left(\begin{array}{l} \left(\begin{array}{l} k - 1 \mapsto i \mapsto j \in dom(d) \\ \wedge k - 1 \mapsto i \mapsto k \in dom(d) \\ \wedge k - 1 \mapsto k \mapsto j \in dom(d) \\ \wedge d(k - 1 \mapsto i \mapsto j) > d(k - 1 \mapsto i \mapsto k) + d(k - 1 \mapsto k \mapsto j) \end{array} \right) \\ \Rightarrow \\ \left(\begin{array}{l} k \mapsto i \mapsto j \in dom(d) \\ \wedge d(k \mapsto i \mapsto j) = d(k - 1 \mapsto i \mapsto k) + d(k - 1 \mapsto k \mapsto j) \end{array} \right) \end{array} \right)$$

- $$axm9 : \forall i, j, k. \left(\begin{array}{l} \left(\begin{array}{l} k - 1 \mapsto i \mapsto j \notin dom(d) \\ \wedge k - 1 \mapsto i \mapsto k \in dom(d) \\ \wedge k - 1 \mapsto k \mapsto j \in dom(d) \end{array} \right) \\ \Rightarrow \\ k \mapsto i \mapsto j \in dom(d) \end{array} \right)$$

INVARIANTS

$inv1 : D \in N \times N \leftrightarrow \mathbb{N}$

$inv2 : FD \in \text{BOOL}$

EVENT shortestpathOK

WHEN

$grd1 : l \mapsto a \mapsto b \in \text{dom}(d)$

THEN

$act1 : D(a \mapsto b) := d(l \mapsto a \mapsto b)$

$act2 : FD := \text{TRUE}$

END

EVENT shortestpathKO

WHEN

$grd1 : l \mapsto a \mapsto b \notin \text{dom}(d)$

THEN

$act1 : FD := \text{FALSE}$

END

```

/* N = 1..n-1 */
void shortestpath (int l, int a, int b, int g[][n], int *D, int *FD)
{
    int D1,D2,D3,FD1,FD2,FD3;

    *FD = 0; FD1=0;FD2=0;FD3=0;
    if (l==0)
    {
        if (g[a][b] != NONE)
        { *FD = 1; *D = g[a][b];}
    }
    else
    {
        shortestpath(l-1,a,b,g,&D1,&FD1);
        if (FD1 == 1) {
            shortestpath(l-1,a,l,g,&D2,&FD2);
            if (FD2==1) {
                shortestpath(l-1,l,b,g,&D3,&FD3);
                if (FD3==1) {
                    if (D1 < D2+D3)
                    { *D= D1;}
                }
                else
                { *D=D2+D3; };
                *FD = 1;}
            else
            { *D=D1;*FD=1; } }
        else
        { *D=D1;*FD=1; } }
    }
    else
    {
        if ( FD2 == 1 && FD3==1) { *D=D2+D3; *FD=1;}
    }
    else
    { *FD=0; }; }
}

```

Model	Total	Auto	Manual	Reviewed	Undischarged
CSPATH1	8	8	0	0	0
SPATH1	5	4	1	0	0
SPATH2	493	317	176	0	0
Global	506	329	177	0	0

- **Proof Obligations are related to d .**
- **The prover provides an effective help for completing the invariant.**

EVENT INITIALISATION

BEGIN

act1 : $m \in \mathbb{N}$

act2 : $temp \in \mathbb{N}$

act3 : $ftemp \in \mathbb{N}$

act4 : $l := start$

END

EVENT maximum($f, n, i; m$)% $i=0$

REFINES maximum

WHEN

grd1 : $l = start$

grd2 : $i = 0$

THEN

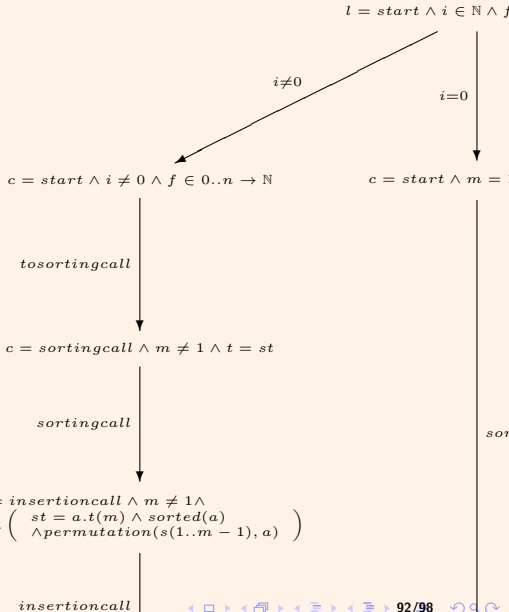
act1 : $l := end$

act2 : $m := f(0)$

END

EVENT ELSE

WHEN



```
EVENT rec%maximum(f,n,i-1;temp)%i/=0
  WHEN
    grd1l = call1
  THEN
    act2l := call2
    act1temp : |(temp' ∈ ran(f) ∧ (∀j·j ∈ 0 .. i - 1 ⇒ temp' ≥ f(j)))
  END
EVENT case1
  WHEN
    grd1l = call2
    grd2f(i) < temp
  THEN
    act2l := call3
    act1ftemp := temp
  END
EVENT case2
  WHEN
    grd1l = call2
    grd2f(i) ≥ temp
  THEN
    act1ftemp := f(i)
    act2l := call3
  END
```

```
EVENT maximum(f,n,i;m)%i/= 0  
REFINES maximum  
WHEN  
  grd1l = call3  
THEN  
  act2l := end  
  act1m := ftemp  
END
```

- **EVENT** $\text{rec}\% \text{proc}(x,y)\% \text{pre}(\text{proc})(x)\% \text{post}(\text{proc})(x,y)$: a general form for naming an event corresponding to a recursive call
- **EVENT** $\text{call}\% \text{aproc}(x,y)\% \text{pre}(\text{aproc})(x)\% \text{post}(\text{aproc})(x,y)$: a general form for naming an event corresponding to a call of `APROC`

- Applying one-step refinement.
- Relating programming and modelling concepts
- Complex algorithms
- Correctness is simpler to get by using the prover.